

Development Plan

Software Engineering

Team 24, Maca Boca
Javayria Mudassar
Yilin Jiang
Nour Fawaz
Catherine Yang

Table 1: Revision History

Date	Developer(s)	Change
Sep 24th, 2026	All	Added first draft version
Date2	Name(s)	Description of changes
...	...	...

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

[State whether your project has confidential information from industry, or not. If there is confidential information, point to the agreement you have in place. —SS]

[For most teams this section will just state that there is no confidential information to protect. —SS]

2 IP to Protect

[State whether there is IP to protect. If there is, point to the agreement. All students who are working on a project that requires an IP agreement are also required to sign the “Intellectual Property Guide Acknowledgement.” —SS]

3 Copyright License

[What copyright license is your team adopting. Point to the license in your repo. —SS]

4 Team Meeting Plan

Weekly meetings will be held between all members on Wednesday at 6:30PM, ideally in person; whether that’s on campus or at a team member’s home. This schedule is flexible to accomodate online meetings, if necessary. Members are expected to have read up on all relevant materials and completed any assigned action items from the previous meeting, to ensure that meets are productive and efficient. All members are encouraged to chair meetings, and meeting minutes will be taken by a designated team member, and shared with the team after each meeting.

Additionally, weekly meetings with the supervisor and other professors involved in the project will be held on Fridays, in order to stay on track with requirements and receive ongoing guidance throughout our deliverables. Each meeting will be recorded and meeting minutes will be taken as well with the same template as general meetings.

Meeting Minutes/Agenda Template

1. Date, Time, Location
2. Attendees
3. Goal of Meeting
4. Agenda items (Review of previous meeting minutes, discussion of any challenges the team has, questions/concerns, and required team decisions)
5. Action Items (Assign responsibilities to each member, note down deadlines and potential blockers between items)
6. **IF a supervisor meeting is upcoming:** note down all questions to be discussed with the supervisor, and any updates that we can provide them.
7. **IF a new deliverable was dropped:** review requirements as a team, brainstorm ideas/thoughts, and assign responsibilities to each member.
8. Notes (Summarize decisions, close meeting)

5 Team Communication Plan

[Issues on GitHub should be part of your communication plan. —SS]

6 Team Member Roles

[You should identify the types of roles you anticipate, like notetaker, leader, meeting chair, reviewer. Assigning specific people to those roles is not necessary at this stage. In a student team the role of the individuals will likely change throughout the year. —SS]

7 Workflow Plan

Git Usage

The team will use Git and GitHub for version control and project management. After being assigned individual tasks, team members are required to create an issue, and to create and work on a new branch. All work will be completed on the branch and submitted for review through a pull request (PR). All members are encouraged to review any open pull request, but at least one team member must review and approve the changes before the user can merge their branch onto main.

Naming Conventions

The project will make use of following naming conventions:

Pull Requests/Commits: `<type><scope>`: `<short description>`. The supported types are:

- **feat**: Adding new functionality
- **fix**: Fixing a bug/error
- **docs**: Updating documentation
- **test**: Adding/updating tests
- **refactor**: Restructuring code, no external behavior change
- **chore**: Github maintenance, builds, dependencies

The scope identifies the primary component or document affected by the change, such as **ui**, **backend**, **data**, or the name of a specific document. The scope can be omitted if no appropriate scope applies.

For example:

```
feat(ui): add activity selection screen
```

Branches: `<type>/<name>/<short-description>`, where `<type>` identifies the purpose of the branch, `<name>` identifies the contributor, and `<short-description>` briefly describes the work being completed. Descriptions should use lowercase words separated by hyphens.

For example:

```
feat/javayria/activity-selection-screen
```

Handling Pull Requests

When opening a pull request, the default template will be applied. It includes a description of the changes, added tests + test scenarios, linked issues, and a checklist. The checklist serves as a reminder for authors to review their work before submitting the pull request for review, but does not block it from being merged. When merging pull requests, the ‘Squash and merge’ option should be selected to help keep *main*’s history clean and readable.

Issue Management

All progress on the team will be tracked via Github Issues and a Kanban board. This includes every lecture and different type of meeting attended, as well as all development tasks and deliverables. Template issues have been created for each type of task/issue, and can be easily updated via configurable files in the repository; currently including sections for dates, descriptions, and relevant notes. Each issue also has a specific label to categorize it (e.g “Lecture” for Lecture Issues, and “Bug” all Development Bug Issues). Each member will be responsible for updating their own issues with relevant pull requests (must be closed by updating the ‘Linked Issues’ section of the PR description), and status changes as the task progresses. The coordinator will also regularly review the project board to ensure that all issues are on track.

Issue Templates:

1. Lecture Issue: To track any relevant notes detailed in lecture.
2. Supervisor Meeting Issue: To track all pre-meeting agenda items, questions, and notes with the supervisor and external teams.
3. Team Meeting Issue: To track all pre-meeting agenda items, questions, and notes for general meetings.
4. TA Meeting Issue: To track all pre-meeting agenda items, questions, and notes with the TA.
5. Deliverable Issue: To track requirements and progress on all deliverable items.
6. Blank Issue: To track any other important items.

CI/CD

To prevent any failures or issues from occurring, we'll be using Github Actions to implement our main CI/CD pipelines. These workflows will then be triggered on every pull request that opens or updates, and again on every merge to main. Each workflow will run the project's integration, contract or unit tests, and check for any formatting or linting errors before allowing the changes to be merged. If any of these builds or runs fail, the team member who opened the pull request must go back and fix the issue before merging. Another workflow will also be added to compile our LaTeX documentation whenever a `.tex` file is changed, ensuring that every document still generates a PDF without any errors. The compiled PDFs will also be uploaded as artifacts so whoever is reviewing the pull request can actually verify the final formatted output before approving. Once every check has passed, a final team member must stamp their approval in order to finally merge the changes into main.

Workflow Summary:

1. Brainstorm and discuss all tasks as a team.
2. Individual task is assigned to a team member, who creates an issue on the kanban board to track.
3. Team member creates a new branch from latest main, to work on their task. They must ensure that they're frequently pruning their branch to the latest main.
4. Any progress or significant changes should be updated on the kanban issue board.
5. Team member regularly commits their changes, with descriptive/clear commit messages that follow the outlined naming conventions.

6. Once the task is complete, the team member submits a pull request, and assigns a reviewer. The pull request description should include be complete, with all template fields filled out and checklists up to date.
7. The reviewer checks the PR, following any testing or tophatting instructions provided, and leaves comments if it needs any changes. If not, they approve the PR with a smiley face.
8. Once the PR is approved, the team member double checks their work and merges their branch into main.
9. Team member closes the issue on the kanban board if it was not done by an accompanying PR.

8 Project Decomposition and Scheduling

We will use GitHub Projects as the single source of truth for documentation and code (link to the GitHub project is [HERE](#)). The board (*Maca Kanba*) contains all project issues, with a Kanban view for day-to-day work and a table/roadmap view grouped by milestone. Every issue has at least one assigned owner to ensure it gets completed, though there may be multiple collaborators if required. Two issue types are used:

1. *Documentation issues*: one per deliverable section, using the documents.md template (Document Name / Description / Due Date / Reviewed by).
2. *Code issues*: one per feature/module/task, using a separate code template.

Milestones

Milestones are grouped by graded deliverable date to help keep the team on track for due dates. Peer-review milestones follow immediately after their corresponding deliverable. For example, if M2 is SRS + Hazard Analysis, then M2.1 is SRS Peer Review and all issues related to addressing feedback are tracked there. Code-heavy sections (POC, Rev 0 module implementation, Rev 1) may get their own milestones separate from any documentation due the same week, since code issues require a separate lifecycle that relies more on testing and thorough reviews.

Project Schedule

- **September 28 – November 19: Requirements**
 - Problem Statement
 - Proof of Concept Plan
 - Development Plan & Team Charter
 - Requirements Document (SRS)

- Hazard Analysis, V&V Plan Revision 0
- Design Document Revision -1 with CD-Fall
- **January 18 – January 29: Proof of Concept & Module Implementation**
 - Design Document Revision 0
 - Scheduling constraint requests
 - Core module implementation
 - POC Demonstration
 - Team Contribution Survey for Revision 0
 - Team Productivity Report for Revision 0
- **(POC/Rev 0 Demo dates) – March 8: Integration & Verification**
 - Integration of completed modules
 - V&V Report
 - CD-Winter
- **March 8 – April 6: Finalization & Expo**
 - Team Contribution Survey for Revision 1
 - Team Productivity Report for Revision 1
 - Source code and Final Documentation for Revision 1
 - Final Demonstration
 - Capstone EXPO

The project can thus be broken down into the following milestones:

Table 2: Project Milestones

Milestone	Deliverable(s)	Due Date
M1	Problem Statement, POC Plan, Development Plan, Team Charter	Sep 28
M1.1	Peer Review: above	Oct 1
M2	SRS + Hazard Analysis	Oct 19
M2.1	Peer Review: SRS + Hazard Analysis	Oct 22
M3	V&V Plan Rev 0	Nov 2
M3.1	Peer Review: V&V Plan	Nov 5
M4	Design Doc Rev -1, CD-Fall	Nov 16
M4.1	Peer Review: Design Doc Rev -1	Nov 19
M5	Scheduling constraint requests	Jan 18
M6	Module Implementation	Jan 21
M6.1	Design Doc peer module reflection	Jan 25
M7	Team Contribution Survey + Productivity Report Rev 0	Jan 29
M8	V&V Report + CD-Winter	Mar 8
M9	Team Contribution Survey + Productivity Report Rev 1	Mar 18
M10	Final Documentation (Rev 1) + Source Code	Mar 31
M11	EXPO Demonstration	Apr 6

9 Proof of Concept Demonstration Plan

What is the main risk, or risks, for the success of your project? What will you demonstrate during your proof of concept demonstration to convince yourself that you will be able to overcome this risk?

10 Expected Technology

[What programming language or languages do you expect to use? What external libraries? What frameworks? What technologies. Are there major components of the implementation that you expect you will implement, despite the existence of libraries that provide the required functionality. For projects with machine learning, will you use pre-trained models, or be training your own model? —SS]

[The implementation decisions can, and likely will, change over the course of the project. The initial documentation should be written in an abstract way; it should be agnostic of the implementation choices, unless the implementation choices are project constraints. However, recording our initial thoughts on implementation helps understand the challenge level and feasibility of a project. It may also help with early identification of areas where project members will

need to augment their training. —SS]

Topics to discuss include the following:

- Specific programming language
- Specific libraries
- Pre-trained models
- Specific linter tool (if appropriate)
- Specific unit testing framework
- Investigation of code coverage measuring tools
- Specific plans for Continuous Integration (CI), or an explanation that CI is not being done
- Specific performance measuring tools (like Valgrind), if appropriate
- Tools you will likely be using?

[git, GitHub and GitHub projects should be part of your technology. —SS]

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

[What coding standard will you adopt? —SS]

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Javayria

If there's one thing I took away from my co-op, it's that structure and planning before starting a project is extremely necessary. We are all software engineers with our own ways of thinking and working, and while individuality is great, there needs to be some pre-agreed upon, formalized set of rules and methodologies for the team to succeed. Having a plan allows the team to focus on what's important rather than getting lost in fine-grained details. It also sets a standard, where everyone is aware of the level of work quality expected of them beforehand, so they can keep that in mind while working. Is it difficult to confront a team member who isn't contributing to the level expected of them? Yes, it is. However, that's much less likely to happen if the conversation happens at the very beginning, since it helps the team find common ground and ensures everyone goes into the project with the same mindset. I feel confident that my team members and I will benefit from having this plan in place as the project grows more and more complex.

One of the biggest advantages of CI/CD is that it helps catch mistakes and issues with the code before it goes to production (or in this case, the main branch). Running and troubleshooting builds manually is an extremely frustrating process (as any software engineer can tell you!), so having a workflow

run a linter or unit tests against new changes before they're ready to be merged is a great first step for verification. One of the major drawbacks, though, is the amount of time it takes to both run and troubleshoot CI/CD pipelines (here, we mean in the context of GitHub). It's very easy to find yourself waiting on a 30-minute build pipeline, only for it to fail midway and need to be run again. This can slow down the coding process and cause PRs to take multiple days to be reviewed and merged. I expect that we will need to tweak/update the workflows throughout the term as we notice certain parts taking too long or repeatedly failing.

Our team did not have any major disagreements when working on this deliverable. I found that everyone was respectful and open to hearing each other's ideas, which is a good sign for the project's future. As we start reviewing each other's work moving forward, we will need to continue being respectful and kind in our communication. If there happen to be any disagreements within the group, then both parties are expected to handle them immediately in an open, respectful manner.

Catherine

Without a development plan to layout the foundation for how our project is structured and run, I think we'd immediately lose our heads when it comes to things like formatting, timelines, and most *importantly* motivation. When things get messy and convoluted, I personally dread tackling it and putting off work as important as our capstone project is the last thing I want to do. The plan is there to make things easier for ourselves as members, as it's just a structured guideline that we can continuously refer to throughout the project. We don't have to continuously reinvent new rules or reconvene to discuss things that we've already done and gone over. As the project progresses, I'm 100% sure things will only become more complex and messy, so having a plan in place from the start is even more important to ensure that we're moving as efficiently and as organized as possible.

I believe that there's a lot of pros to having CI/CD integrated in your project. If there's any test-breaking code, linting errors that create rendering or deployment errors, or stale code from bad merges/previous commits; having a pipeline to catch everything *before* they go into main can ultimately save a lot of headache everybody involved. As now maintenance can happen while the new changes are contained in a pull request. On the downside though, they can take a while to setup. I expect that we'll have to set aside additional time for the testing process of any additional pipelines we want to create.

Finally, I don't believe we had any team disagreements for this deliverable. We had a pretty straightforward and productive meeting on our next steps, brainstorming collectively for each section item, before we individually assigned tasks and broke off to work on them separately. We agreed though that if we have any future disagreements, the members involved would communicate

respectfully and sort it out together before moving forward with any work.

Appendix — Team Charter

[borrows from University of Portland Team Charter —SS]

External Goals

For this capstone project, our team’s external goals are to achieve an A+ first and foremost, but to also have a finished product that actually demonstrates what we’ve learned about healthcare in a speech-language pathology setting, and also how technology can actually reduce barriers that many in the field face, from both clinicians and patients. We hope to have a product that future employers can utilize, and also showcase on our resumes to show off our skillset as engineers.

Attendance

Expectations

[What are your team’s expectations regarding meeting attendance (being on time, leaving early, missing meetings, etc.)? —SS]

Acceptable Excuse

[What constitutes an acceptable excuse for missing a meeting or a deadline? What types of excuses will not be considered acceptable? —SS]

In Case of Emergency

[What process will team members follow if they have an emergency and cannot attend a team meeting or complete their individual work promised for a team deliverable? —SS]

Accountability and Teamwork

Quality

Team members are expected to spend 20-30 minutes reviewing relevant documents before each meeting, or enough to contribute meaningfully and raise questions or notes as needed. Before meetings with the supervisor, all members must be fully aware of the project’s status and prepared to both ask and answer questions.

For deliverables, each member is responsible for completing their assigned section to the best of their ability: original content, free of spelling and grammar errors, and going beyond the bare minimum. If a section is entirely AI-generated, the author must be able to explain the writing/code and prove to have a deep understanding of the work they put out. All team members are responsible for properly testing their code with the appropriate endpoints to ensure any existing

functionality is not broken. Each section should be checked against the Avenue rubric to confirm all requirements are met, and reviewed by a designated team member before submission.

All deliverables must be finalized two days before the official deadline to allow time for review. Any issues found afterward are to be fixed by the original author. Final submission occurs by 7:00 pm on the due date to allow time for technical issues.

Attitude

[What are your team's expectations regarding team members' ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment with their TA, or the instructor? Are there incentives for reaching targets early? —SS]

Team Building

[How will you build team cohesion (fun time, group rituals, etc.)? —SS]

Decision Making

[How will you make decisions in your group? Consensus? Vote? How will you handle disagreements? —SS]